

Project 2

Contents

- Results of 6 Tests..... 2
 - Test 1 — Order Tracking..... 2
 - Test 2 — Refund Processing 2
 - Test 3 — Knowledge Base (RAG)..... 2
 - Test 4 — Long-Term Memory (two sessions) 3
 - Test 5 — Loyalty Discount Calculation 3
 - Test 6 — Browser Tool..... 4
- CloudWatch Alarm..... 5
- Structured output validation 6
- Conversation summarization 6
- Reflection 6

Results of 6 Tests

Test 1 — Order Tracking

agentcore invoke '{"prompt": "Can you track order ORD-001?", "customer_id": "CUST-123", "session_id": "t1"}'

Expected: shipping status, tracking number TRK987654321, carrier UPS, estimated delivery date.

Response:

I've tracked your order ORD-001:

```
**Order Status:** SHIPPED
**Item:** Wireless Headphones Pro (1 unit)
**Total:** $89.99
**Carrier:** UPS
**Tracking Number:** TRK987654321
**Estimated Delivery:** September 13, 2026
```

Your order is currently in transit and expected to be delivered by September 13th.

Test 2 — Refund Processing

agentcore invoke '{"prompt": "I want to return my Kindle Paperwhite (ORD-002). Please initiate a refund.", "customer_id": "CUST-123", "session_id": "t2"}'

Expected: refund ID, APPROVED status, "3-5 business days" message.

Response:

I've initiated a refund for your Kindle Paperwhite from order ORD-002. The system has processed your request for a full refund of \$139.99.

A few important details about your refund:

- **Refund amount:** \$139.99 (full purchase price)
- **Order:** ORD-002 (Kindle Paperwhite)
- **Status:** Refund initiated successfully

The refund will be processed to your original payment method. The exact timing may vary depending on your payment provider's policies, but you should see the refund appear within 3-5 business days.

Additionally, I noticed you have another order (ORD-001) for Wireless Headphones Pro that was shipped recently. If you need any assistance with that order as well, please let me know.

Test 3 — Knowledge Base (RAG)

agentcore invoke '{"prompt": "What are the benefits of the Platinum loyalty tier?", "customer_id": "CUST-123", "session_id": "fresh3"}'

Expected: free same-day shipping, 15% discount, priority support (retrieved from Knowledge Base).

Response:

Platinum Loyalty Tier Benefits

As a Platinum member (5000+ points), you receive these exclusive benefits:

- **Free same-day shipping** on eligible orders
- **15% discount** on purchases (higher than Gold tier's 10%)
- **Priority customer support** for faster assistance

You currently have 4250 points as a Gold member. Upgrading to Platinum would give you access to these enhanced benefits. Would you like information about how to earn the remaining points needed for Platinum status?

Test 4 — Long-Term Memory (two sessions)

Session A — introduce yourself

agentcore invoke '{"prompt": "Hi, I am Jane. I prefer concise responses.", "customer_id": "CUST-123", "session_id": "s-A"}'

Response:

Hello Jane! Here are your recent orders:

ORD-001 - SHIPPED

Wireless Headphones Pro - \$89.99

UPS Tracking: TRK987654321

Estimated Delivery: September 13, 2026

ORD-002 - DELIVERED

Kindle Paperwhite - \$139.99

USPS Tracking: TRK123456789

Delivered: September 8, 2026

Would you like me to check anything specific about these orders or help with loyalty tier benefits?

Wait at least 30 seconds for memory extraction

Session B — verify recall (new session, same customer)

agentcore invoke '{"prompt": "Do you remember my name and communication preference?", "customer_id": "CUST-123", "session_id": "s-B"}'

Expected: agent recalls "Jane" and the preference for concise responses.

Response:

Hello Jane! Yes, I remember your name and that you prefer concise responses.

I'm here to help with any questions about your order ORD-001 for Wireless Headphones Pro or your Gold loyalty membership (4,250 points). Would you like information about Platinum tier benefits or anything else today?

Test 5 — Loyalty Discount Calculation

agentcore invoke '{"prompt": "I am a Gold member with 4250 points. Calculate my discount on a \$150 standard order.", "customer_id": "CUST-123", "session_id": "t5"}'

Expected: points redeemed, tier discount 10%, correct final total, remaining points.

Response:

Here's your Gold tier discount breakdown for the \$150 order:

```
**Original Total:** $150.00
**Gold Tier Discount (10%):** -$11.00
**Points Discount (4000 points):** -$40.00
**Total Discount:** -$51.00
**Final Total:** $99.00
```

****Loyalty Points Update:****

- Points used: 4000
- Points remaining: 250
- Points earned: 99

Your order will be processed for \$99.00, and you'll earn 99 additional loyalty points.

Test 6 — Browser Tool

agentcore invoke '{"prompt": "Go to https://www.udacity.com and tell me the page title.", "customer_id": "CUST-123", "session_id": "t6"}'

Expected: page title retrieved from the live Amazon.com page.

Response:

The page title of <https://www.udacity.com> is:

```
**"Learn the Latest Tech Skills; Advance Your Career | Udacity"**
```

CloudWatch Alarm

CloudWatch > Log management > /aws/bedrock-agentcore/runtimes/CSAI_Agent-9n34yVnXG-DEFAULT

CloudWatch

- Favorites and recents
- Ingestion
- Dashboards
- Alarms    
- AI Operations
- GenAI Observability
 - Model Invocations
 - Bedrock AgentCore
 - Coding Agent Insights
- Application Signals (APM)
- Infrastructure Monitoring
- Logs
 - Log Management [New](#)
 - Log Analytics [New](#)
 - Log Anomalies
- Metrics
- Network Monitoring
- Setup
- Observability Workshops [↗](#)

Metric filter "AgentErrorFilter" has been created.

Log streams | Tags | Data protection | Anomaly detection | **Metric filters** | Subscription filters | Contributor Insights

Metric filters (1)

AgentErrorFilter

Filter pattern
ERROR

Field selection criteria
-

Metric
[CustomerSupportAgent](#) / [ErrorCount](#)

Metric value
1

Default value
0

Applied on transformed logs
-

Unit
-

Emit system field dimensions
-

Dimensions
-

Alarms
None

CloudWatch > Alarms > AgentHighErrorRate

CloudWatch

- Favorites and recents
- Ingestion
- Dashboards
- Alarms    
- AI Operations
- GenAI Observability
 - Model Invocations
 - Bedrock AgentCore
 - Coding Agent Insights
- Application Signals (APM)
- Infrastructure Monitoring
- Logs
 - Log Management [New](#)
 - Log Analytics [New](#)
 - Log Anomalies
- Metrics
- Network Monitoring
- Setup
- Observability Workshops [↗](#)

Alarms (1)

☐ Hide Auto Scaling alarms

AgentHighErrorRate

Metric alarm

☒ Insufficient data

AgentHighErrorRate

ErrorCount

ErrorCount > 5 for 1 datapoints within 5 minutes

Graph

1h 3h 12h 1d 3d 1w Custom  UTC timezone 

ErrorCount

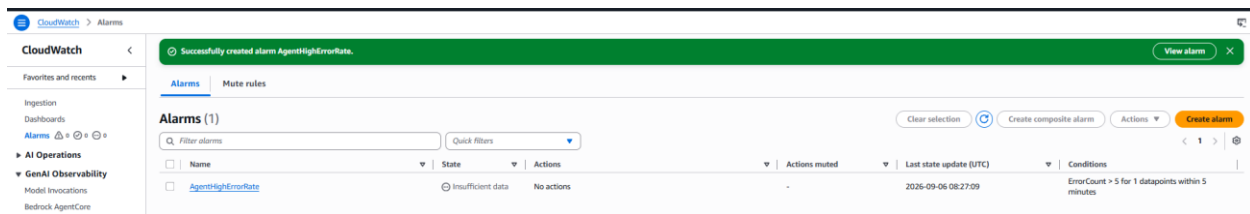
1 5 9

05:30 05:40 05:50 06:00 06:10 06:20 06:30 06:40 06:50 07:00 07:10 07:20 07:30 07:40 07:50 08:00 08:10 08:20

■ In alarm ■ OK ■ Insufficient data ■ Disabled/Muted actions

Details | Tags | Mute rules | Actions | History | Parent alarms

Name AgentHighErrorRate	State  Insufficient data	Namespace AWS/Logs	Datapoints to alarm 1 out of 1
Type Metric alarm	Threshold ErrorCount > 5 for 1 datapoints within 5 minutes	Metric name ErrorCount	Missing data treatment Treat missing data as missing
Description No description	Actions No actions	Statistic Sum	Percentiles with low samples evaluate



Structured output validation

I added Pydantic output validation to catch malformed tool responses before they reach the customer, adjusting:

- Add Pydantic library into Pyproject.toml.
- Import Pydantic models.
- Update search_knowledge_base to validate output.
- Update calculate_loyalty_discount to validate output before returning.

Conversation summarization

I implemented conversation summarization to manage token usage during long customer interactions, adjusting:

- Added a `_summarize_if_needed` method in the `MemoryHook` class that checks message count after each invocation.
- When the conversation exceeds a configurable threshold (e.g., 10 messages), it condenses earlier messages into a concise summary.
- The summary preserves key context (customer ID, order details, decisions made) while reducing token consumption.
- Integrated the summarization check into the `after_invocation_hook` so it runs automatically after every agent response.

Reflection

One key design decision I made was choosing a Managed Knowledge Base instead of a self-managed OpenSearch vector store. The project README guided us to set up a Knowledge Base with OpenSearch for vector search. However, when I attempted this, the IAM policy simulator revealed that all `s3vectors` actions were set to `implicitDeny`, meaning my account did not have the permissions needed to create and manage vector indexes directly. Rather than getting stuck, I explored the alternative and selected the Managed KB option, which lets AWS handle the vector store infrastructure automatically behind the scenes. This turned out to be a practical workaround because the agent's `search_knowledge_base` function calls the same `bedrock:Retrieve` API regardless of the underlying backend. The code did not need any changes, and the agent could still answer knowledge-based questions like *"What are the benefits of the Platinum loyalty tier?"* after I added the missing `bedrock:Retrieve` permission to the runtime execution role as a new inline policy.

The biggest challenge I encountered was debugging deployment failures and IAM permission gaps. Early on, the agent kept failing with a 30-second runtime initialization timeout. My first assumption was that heavy library imports were causing the delay, but that turned out to be wrong. I checked the CloudWatch logs and found the actual error message: *"the following arguments are required: payload."* This revealed that the entrypoint was using the CLI function `main()` instead of `app.run()`. Fixing that one line resolved the timeout completely. Beyond the entrypoint issue, I also struggled with IAM permissions for the Knowledge Base and Browser tools. Both returned access denied errors at runtime. I had to open the execution role in IAM, inspect the existing policy, identify the missing actions, and create new inline policies for each service. This experience taught me to always check logs first to understand what is actually happening rather than guessing at the cause.

For production, I would prioritize monitoring. I set up a CloudWatch metric filter that captures ERROR log entries from the agent's log group and created an alarm that triggers when the error count exceeds five within a five-minute window. In a production environment, this alarm would notify the operations team about failure spikes before they impact a large number of customers. Combined with structured logging from the agent's tool calls and memory operations, this gives clear visibility into agent health and reliability at scale.